Zagadnienia 12, 24, 6B

Mateusz Pękalak

Pochodna funkcji wielu zmiennych

- Pochodna cząstkowa
- Pochodna kierunkowa
- Pochodne wyższych rzędów
- Macierz Hessego i warunki na ekstremum
- Ekstrema warunkowe — mnożniki Lagrange'a
- Warunki konieczne i dostateczne istnienia ekstremów

Pochodna cząstkowa

- Pochodna cząstkowa to pojęcie, które pozwala na badanie, jak funkcja zmienia się względem jednej zmiennej, traktując pozostałe zmienne jako stałe. Jest to bardzo ważny temat w analizie funkcji wielu zmiennych.

Pochodna w określonym punkcie $\lim_{\Delta x \to 0} \frac{f(x_0+\Delta x)-f(x_0)}{\Delta x}$

Pochodna w dowolnym punkcie $\lim_{\Delta x \to 0} \frac{f(x+\Delta x)-f(x)}{\Delta x}$

Przydatne wzory

1. $(C)' = 0$
2. $(x^n)' = nx^{n-1}$
3. $(x)' = 1$
4. $\left(\frac{1}{x}\right)' = -\frac{1}{x^2}$
5. $(\sqrt{x})' = \frac{1}{2\sqrt{x}}$
6. $(a^x)' = a^x \ln a$
7. $(e^x)' = e^x$
8. $(\log_a x)' = \frac{1}{x \ln a}$
9. $(\ln x)' = \frac{1}{x}$
10. $(\sin x)' = \cos x$
11. $(\cos x)' = -\sin x$
12. $(tgx)' = \frac{1}{\cos^2 x}$

1. $[f(x)+g(x)]' = f'(x)+g'(x)$
2. $[f(x)-g(x)]' = f'(x)-g'(x)$
3. $[a \cdot f(x)]' = a \cdot f'(x)$
4. $[f(x) \cdot g(x)]' = f'(x)g(x)+f(x)g'(x)$
5. $\left[\frac{f(x)}{g(x)}\right]' = \frac{f'(x)g(x)-f(x)g'(x)}{[g(x)]^2}$

<u>Wzory przydatne w liczeniu pochodnych:</u>

$\sqrt[b]{x^a} = x^{\frac{a}{b}}$

$\frac{1}{x^a} = x^{-a}$

13. $(ctgx)' = -\frac{1}{\sin^2 x}$
14. $(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$
15. $(\arccos x)' = -\frac{1}{\sqrt{1-x^2}}$
16. $(arctgx)' = \frac{1}{x^2+1}$
17. $(arcctgx)' = -\frac{1}{x^2+1}$

Pochodna kierunkowa

- Pochodna kierunkowa jest uogólnieniem pojęcia pochodnej cząstkowej na dowolne kierunki, przy czym pochodne cząstkowe są tożsame z pochodnymi w kierunkach wektorów jednostkowych bazy układu współrzędnych.

- Jeżeli istnieje gradient funkcji $\nabla f(x)$ w punkcie x

$$\text{Gradient } \nabla f = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n}\right]$$

to pochodna kierunkowa funkcji f w kierunku wektora u jest równa iloczynowi skalarnemu gradientu funkcji f i wektora u.

$$\nabla_u f(x) = \nabla f(x) * u$$

Przykład

1. Niech będzie dana funkcja $f(x,y) = x^2 + xy - y^2$
2. Gradient funkcji f wynosi $\nabla f(x,y) = \left[\frac{\partial f(x,y)}{\partial x}, \frac{\partial f(x,y)}{\partial y}\right]$ = $[2x + y, x - 2y]$
3. Pochodna kierunkowa funkcji f w kierunku jednostkowego wektora $u = \left[\frac{1}{\sqrt{5}}, \frac{2}{\sqrt{5}}\right]$ dana jest zależnością

$$\nabla_u f(x,y) = \nabla f(x,y) * u = [2x + y, x - 2y]\left[\frac{1}{\sqrt{5}}, \frac{2}{\sqrt{5}}\right],$$

Czyli $\nabla_u f(x,y) = \frac{1}{\sqrt{5}}(2x + y) + \frac{2}{\sqrt{5}}(x - 2y) = \frac{4x-3y}{\sqrt{5}}$

Pochodne wyższych rzędów

Pochodna wyższego rzędu funkcji to pochodna uzyskana poprzez wielokrotne zastosowanie operacji różniczkowania. Formalnie, pochodna n-tego rzędu funkcji f oznacza wynik zastosowania operacji różniczkowania n razy względem zmiennej niezależnej.

Dla funkcji f(x)

- f'(x) to pochodna pierwszego rzędu,
- f''(x) to pochodna drugiego rzędu (pochodna pochodnej f'(x)),

Przykłady

- Funkcja $f(x) = x^3$

  Pierwsza pochodna: $f'(x) = 3x^2$

  Druga pochodna: $f''(x) = 6x$

  Trzecia pochodna: $f'''(x) = 6$

  Czwarta pochodna: $f''''(x) = 0$

- Funkcja $f(x) = \sin(x)$

  Pierwsza pochodna: $f'(x) = \cos(x)$

  Druga pochodna: $f''(x) = -\sin(x)$

  Trzecia pochodna: $f'''(x) = -\cos(x)$

  Czwarta pochodna: $f''''(x) = \sin(x)$ (funkcja powraca do postaci początkowej)

Ekstremum lokalne funkcji

- Ekstremum lokalne funkcji wielu zmiennych to minimum lub maksimum lokalne, czyli punkt, w którym funkcja przyjmuje wartość mniejszą lub większą niż w sąsiadujących punktach. Aby znaleźć ekstrema, oblicza się pochodne cząstkowe, przyrównuje je do zera i rozwiązuje otrzymany układ równań, aby znaleźć punkty stacjonarne. Następnie, za pomocą macierzy Hessego i jej wyznacznika, bada się charakter tych punktów (czy są to minima, maksima, czy też punkty siodłowe.

Rozwiązywanie krok po kroku

1. Wyznaczanie punktów krytycznych

Liczymy pochodne cząstkowe funkcji f(x,y) względem każdej zmiennej: $f_x(x,y) = \frac{\partial f}{\partial x},\ f_y(x,y) = \frac{\partial f}{\partial y}$

Rozwiązujemy układ równań:

$$f_x(x,y) = 0,\ f_y(x,y) = 0$$

Aby znaleźć punkty krytyczne (potencjalne miejsca ekstremów)

Rozwiązywanie krok po kroku

2. Warunki Hessego

   Liczymy pochodne cząstkowe drugiego rzędu

   $$f_{xx} = \frac{\partial^2 f}{\partial x^2}, f_{yy} = \frac{\partial^2 f}{\partial y^2}, f_{xy} = \frac{\partial^2 f}{\partial x \partial y}$$

   Obliczamy wyznacznik Hessego w punkcie krytycznym (x_0, y_0):

   $$D(x_0, y_0) = f_{xx}(x_0, y_0) \ * f_{yy}(x_0, y_0) \ - (f_{xy}(x_0, y_0))^{\ 2}$$

Rozwiązywanie krok po kroku

3. Ocena rodzaju punktu krytycznego

- Jeśli $D(x_0, y_0) > 0 \; i \; f_{xx}(x_0, y_0) > 0$

$-minimum\ lokalne$

- Jeśli $D(x_0, y_0) > 0 \; i \; f_{xx}(x_0, y_0) < 0$

$-maksimum\ lokalne$

- Jeśli $D(x_0, y_0) < 0 - punkt\ siodłowy$
- Jeśli $D(x_0, y_0) = 0$

$-test\ nie\ rozstrzyga\ rodzaju\ punktu\ krytycznego$

Przykład

Załóżmy, że mamy funkcje

$$f(x,y) = x^2 + y^2 - 4x - 6y + 13$$

1. Liczymy pochodne cząstkowe

$$f_x = 2x - 4, \quad f_y = 2y - 6$$

Rozwiązujemy układ równań:

$$2x - 4 = 0 \Rightarrow x = 2$$

$$2y - 6 = 0 \Rightarrow y = 3$$

Punkt krytyczny: (2,3).

Przykład

2. Liczymy pochodne drugiego rzędu

$$f_{xx} = 2,\ \ f_{yy} = 2, f_{xy} = 0$$

Obliczamy Hesjan:

$$D = (2)(2) - (0)^2 = 4$$

3. Analiza

$$D = 4 > 0$$

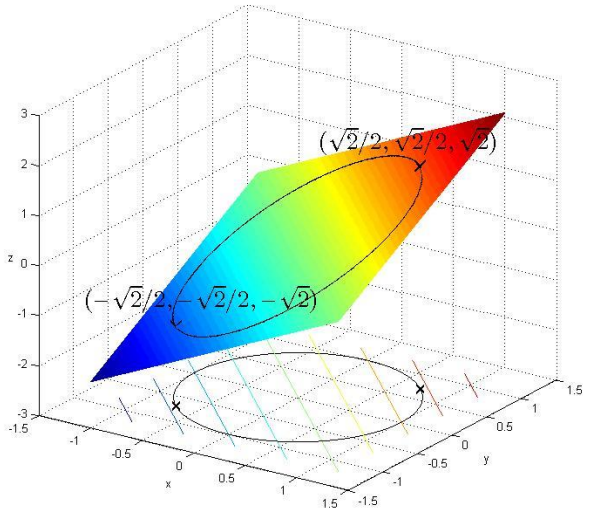
$$f_{xx} = 2 > 0, więc\ mamy\ minimum\ lokalne\ w\ punkcie\ (2,3).$$

Mnożnik Lagrange'a

- Mnożnik Lagrange'a to metoda znajdowania ekstremów funkcji wielu zmiennych, z ograniczeniami.

Gdy mamy funkcję f (x,y,z,...), którą chcemy maksymalizować lub minimalizować, ale nie możemy zmieniać x, y, z dowolnie – muszą one spałniać jakieś ograniczenie w postaci równania g(x, y, z, ...) = 0 – wtedy używamy mnożników Lagrange'a.

Mnożnik Lagrange'a jest oznaczany λ



Używanie mnożnika Lagrange'a

1. Budujemy funkcję Lagrange'a

   Tworzymy nową funkcję:

   $$L(x, y, \lambda) = f(x, y) + \lambda g(x, y)$$

   Poprawnie też można zapisać funkcje z -

   2. Obliczamy pochodne cząstkowe i układamy układ równań

   Do policzenia mamy:

   $$\frac{\partial L}{\partial x} = 0, \frac{\partial L}{\partial y} = 0, \frac{\partial L}{\partial \lambda} = 0,$$

   Trzy równania do rozwiązania – jedno dla każdej zmiennej i jedno dla ograniczenia

Używanie mnożnika Lagrange'a

Rozwiązywanie układu równań

Szukamy x, y, λ

Wartości x, y to kandydaci na miejsca ekstremów (minimum lub maksimum) przy danym ograniczeniu.

Techniki zapewniana jakości w poszczególnych fazach tworzenia oprogramowania

- Inżynieria wymagań
  1. Pozyskiwanie wymagań
  2. Analiza wymagań
  3. Specyfikacja wymagań
  4. Walidacja wymagań
  5. Zarządzanie wymaganiami
- Proces testowania
  1. Planowanie testów
  2. Projektowanie przypadków testowych
  3. Przygotowanie środowiska testowego
  4. Wykonanie testów
  5. Rejestracja i raportowanie błędów
  6. Retesty i testy regresyjne
  7. Zamknięcie testów

Inżynieria wymagań

- Inżynieria wymagań to proces zbierania, analizowania, dokumentowania i zarządzania wymaganiami dotyczącymi systemu informatycznego. Celem jest zapewnienie, że budowane oprogramowanie spełni realne potrzeby użytkowników.

- Inżynierie wymagań podzielimy na 5 kroków.
  1. Pozyskiwanie wymagań
  2. Analiza wymagań
  3. Specyfikacja wymagań
  4. Walidacja wymagań
  5. Zarządzanie wymaganiami

Pozyskiwanie wymagań

Cel: Zebrać informacje, czego naprawdę oczekują użytkownicy i klienci.

Jak to wygląda w rzeczywistości?

- Wywiady z klientami lub użytkownikami końcowymi.
  (np. Product Owner rozmawia z klientem: „Jakie funkcje powinien mieć nowy sklep internetowy?”)
- Warsztaty – spotkania zespołowe (np. Design Thinking Workshop), aby wspólnie ustalić wymagania.
- Ankiety / formularze – szczególnie w projektach masowych (np. tworzenie aplikacji dla 1000 użytkowników).
- Analiza istniejących systemów – jeśli jest już podobne oprogramowanie, patrzy się, co działa, a co nie.

Przykład:
Firma robi nową aplikację bankową. Zespół UX organizuje warsztaty z klientami banku, podczas których zbierają pomysły na funkcje np.: szybki przelew, logowanie przez odcisk palca.

Analiza wymagań

Cel: Uporządkować, wyczyścić i doprecyzować wymagania.

Jak to wygląda w rzeczywistości?

- Grupowanie wymagań na kategorie: funkcjonalne (co system robi) i niefunkcjonalne (wydajność, bezpieczeństwo).
- Wyszukiwanie sprzeczności – np. jeden klient chce, aby aplikacja była prosta, a inny – aby miała dużo funkcji.
- Priorytetyzacja wymagań – ustalanie, co jest kluczowe na start, a co może poczekać.

Przykład:
W projekcie aplikacji e-commerce analityk znajduje sprzeczność: marketing chce rejestracji przez Facebooka, a dział bezpieczeństwa się temu sprzeciwia. Analityk organizuje spotkanie, żeby wypracować kompromis (np. rejestracja przez Facebooka + dodatkowe zgody użytkownika).

Specyfikacja wymagań

Cel: Formalne zapisanie wymagań w dokumentach lub narzędziach.

Jak to wygląda w rzeczywistości?

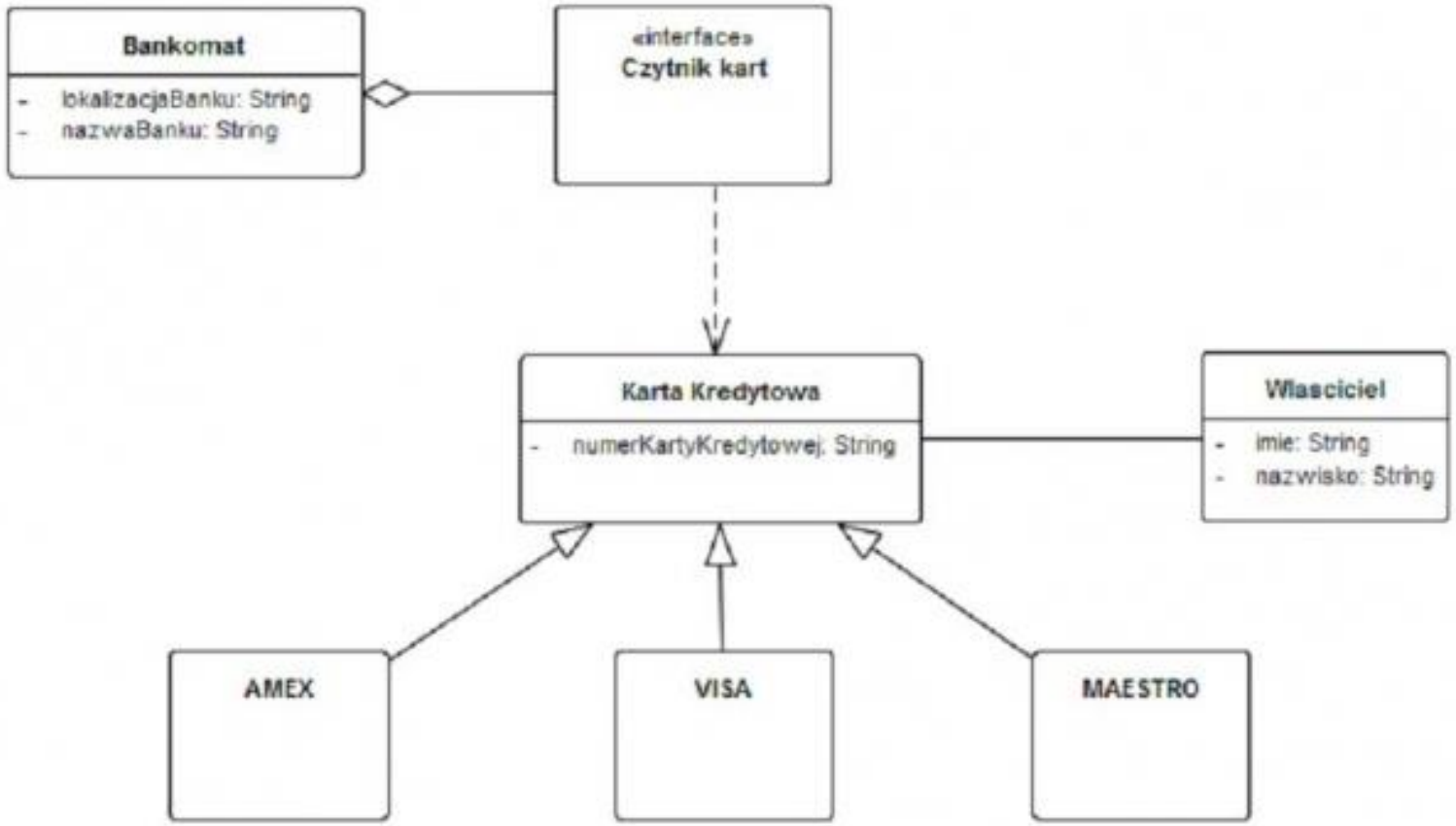
- Tworzenie SRS (Software Requirements Specification) – dokument opisujący wymagania.
- Rysowanie diagramów UML – np. przypadki użycia, diagramy przepływu procesów biznesowych.
- Pisanie user stories i akceptowalnych kryteriów (szczególnie w projektach Agile).

Przykład:
W projekcie platformy edukacyjnej analityk pisze User Story:
„Jako student chcę mieć możliwość przeglądania historii moich ukończonych kursów, aby śledzić mój postęp.”

Akceptacja:

- Historia kursów jest dostępna po zalogowaniu.
- Kursy są uporządkowane od najnowszych do najstarszych

Diagramy UML



Walidacja wymagań

Cel: Upewnić się, że wymagania są poprawne, kompletne i zgodne z oczekiwaniami klientów.

Jak to wygląda w rzeczywistości?

- Przeglądy wymagań z zespołem i klientami – wspólne spotkania, gdzie omawia się dokumenty/specyfikacje.
- Prototypowanie – tworzenie makiet (np. w Figma) i pokazywanie ich klientom przed kodowaniem.
- Testowanie wymagań – sprawdzanie, czy wymagania są mierzalne, osiągalne i testowalne.

Przykład:
UX designer tworzy prototyp nowej aplikacji do zamawiania jedzenia. Pokazuje go na spotkaniu klientowi, który zauważa brak ważnej funkcji – dodawania kilku adresów dostaw. Wymaganie zostaje dopisane do dokumentacji przed rozpoczęciem kodowania.

Zarządzanie wymaganiami

Cel: Utrzymać kontrolę nad wymaganiami w trakcie projektu (bo zmieniają się!).

Jak to wygląda w rzeczywistości?

- Śledzenie zmian – każda zmiana wymagań musi być zarejestrowana i zatwierdzona.
- Version Control – dokumentacja wymagań jest wersjonowana (np. w Confluence, Git).
- Impact Analysis – jeśli wymaganie się zmienia, analizuje się, jakie to ma skutki dla innych funkcji.

Przykład:
Podczas budowy aplikacji mobilnej klient zmienia wymaganie: zamiast loginu i hasła, chce logowanie biometryczne (odcisk palca). Analityk tworzy nowe User Story, aktualizuje backlog i robi analizę wpływu na backend, API i bezpieczeństwo.

Proces testowania

- Proces testowania to zestaw działań mających na celu wykrycie błędów i sprawdzenie jakości tworzonego oprogramowania. Testowanie pozwala upewnić się, że produkt działa poprawnie i spełnia wymagania.

- Proces testowania podzielimy na 7 kroków.
    1. Planowanie testów
    2. Projektowanie przypadków testowych
    3. Przygotowanie środowiska testowego
    4. Wykonanie testów
    5. Rejestracja i raportowanie błędów
    6. Retesty i testy regresyjne
    7. Zamknięcie testów

Planowanie testów

Cel: Zaplanować co będziemy testować, jak, czym i kto to zrobi.

Jak to wygląda w rzeczywistości?

- Tworzy się dokument Plan Testów (Test Plan).
- Ustala się zakres testów – co będzie testowane, a co nie.
- Wybiera się typy testów – funkcjonalne, wydajnościowe, bezpieczeństwa, itd.
- Definiuje się zasoby – ile testerów, jakie urządzenia, jakie narzędzia.

Przykład:
Dla aplikacji mobilnej banku planujemy:

- Testy funkcjonalne (przelewy, historia konta),
- Testy bezpieczeństwa (logowanie biometryczne),
- Testy wydajności (czas ładowania aplikacji poniżej 2 sekund),
- 2 testerów manualnych + 1 tester automatyzacji,
- Testowanie na iPhone 13, Samsung Galaxy S21 i emulatorze.

Projektowanie testów

Cel: Opracować konkretne przypadki testowe (co i jak sprawdzać).

Jak to wygląda w rzeczywistości?

- Tworzy się przypadki testowe (Test Cases) i scenariusze testowe (Test Scenarios).
- Przygotowuje się dane testowe (np. fikcyjni użytkownicy, produkty).
- Opisuje się kroki testowe i oczekiwane wyniki.

Przykład:
Projektujemy przypadek testowy dla logowania:

- Wejdź na stronę logowania, wpisz poprawne dane, kliknij "Zaloguj", sprawdź, czy użytkownik zostaje przeniesiony na stronę główną.
- Projektujemy też przypadki dla błędnych haseł, pustych pól, wygaśnięcia sesji itd.

Przygotowanie środowiska testowego

Cel: Przygotować systemy, narzędzia i dane, żeby móc rozpocząć testowanie.

Jak to wygląda w rzeczywistości?

- Instalacja aplikacji na serwerach testowych lub urządzeniach.
- Konfiguracja baz danych z danymi testowymi.
- Instalacja narzędzi testowych (np. Postman, Selenium, TestRail).
- Przygotowanie kont testowych (admin, użytkownik standardowy).

Przykład:
Przed testowaniem nowego sklepu internetowego:

- Stawiamy środowisko testowe https://test.mojsklep.pl,
- Instalujemy kopię aplikacji i bazę danych z fikcyjnymi produktami,
- Tworzymy 10 kont użytkowników do różnych scenariuszy zakupów.

Wykonanie testów

Cel: Wykonać testy według zaplanowanych przypadków i zebrać wyniki.

Jak to wygląda w rzeczywistości?

- Testowanie według przygotowanych przypadków testowych.
- Rejestrowanie rezultatów: test przeszedł (Pass) lub nie przeszedł (Fail).
- Rejestrowanie błędów (Bug Reporting) w systemach typu Jira.

Przykład:
Tester wykonuje przypadek testowy logowania:

- Test "logowanie poprawne" – Pass.
- Test "logowanie pustym hasłem" – Fail (aplikacja zawiesza się).
- Zgłasza błąd w Jira: „Aplikacja zawiesza się przy pustym haśle podczas logowania – krytyczny.”

Rejestracja i raportowanie błędów

Cel: Zgłaszać i opisywać błędy znalezione w testach.

W praktyce:

•Rejestracja błędów w systemach typu Jira, Azure DevOps.

•Opisanie: gdzie błąd występuje, jakie były kroki do jego odtworzenia.

•Określanie priorytetów (np. Krytyczny, Wysoki, Średni, Niski).

Przykład:
Zgłoszenie do Jira:

•Tytuł: "Nieprawidłowa walidacja adresu e-mail".

•Kroki: Wprowadzono abc@com, zaakceptowano.

•Priorytet: Wysoki.

Retesty i testy regresyjne

Cel: Sprawdzić poprawki oraz upewnić się, że nic innego nie zostało zepsute.

W praktyce:

- Retest: Sprawdzamy błędy po poprawce.
- Test regresyjny: Testujemy, czy zmiany nie popsuły innych funkcji.

Przykład:
Poprawka walidacji e-maila:

- Retestujemy poprawkę (czy teraz odrzuca abc@com).
- Test regresyjny – sprawdzamy inne formularze rejestracji.

Zakończenie testów

Cel: Podsumować pracę, zamknąć proces testowy i przygotować raporty.

Jak to wygląda w rzeczywistości?

- Tworzy się Raport z Testów (Test Summary Report).
- Analizuje się jakie testy przeszły, ile było błędów, jakie są otwarte błędy.
- Organizuje się spotkanie podsumowujące z zespołem developerskim i klientem.
- Archiwizuje się przypadki testowe, środowiska, dane.

Przykład:
Po zakończeniu testowania nowej funkcji w aplikacji mobilnej banku:

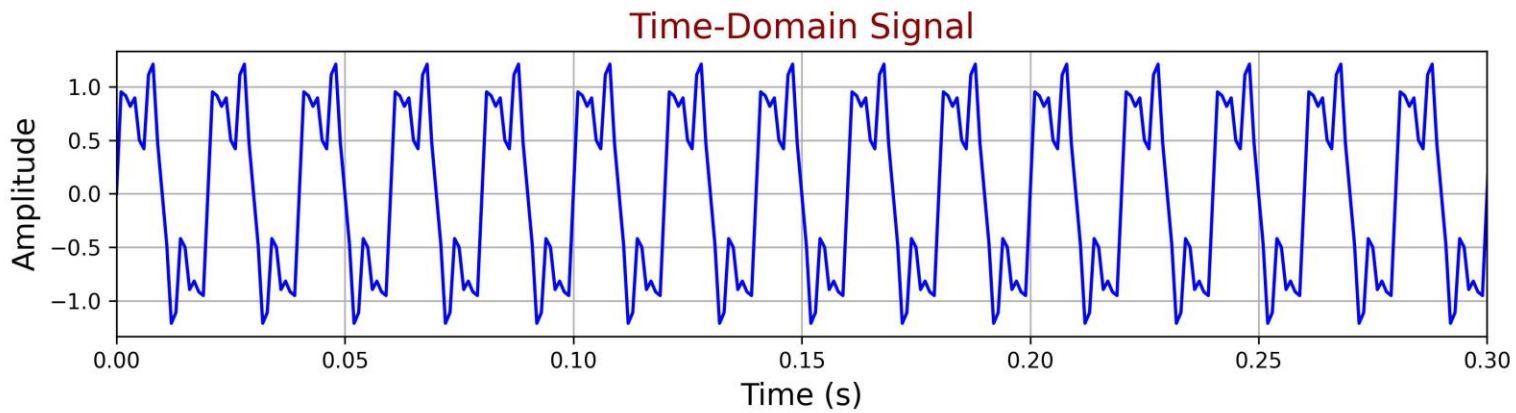
- 50 przypadków testowych -> 47 Pass, 3 Fail.
- 3 otwarte błędy – uzgodnione jako „do poprawienia w kolejnym wydaniu”.
- Przygotowany raport, z którym przechodzi się na spotkanie retrospektywne.

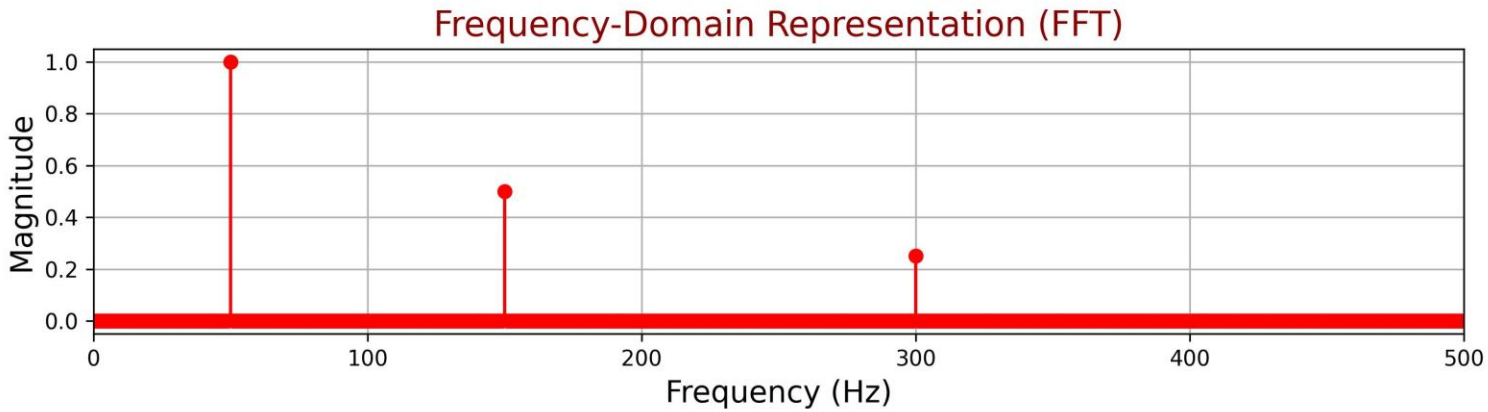
Transmormata Fouriera i jej zastosowania

- Definicja szerego Fouriera i transformacji Fouriera
- DFT
- FFT
- Przykłady zastosowań

Transformata Fouriera

- Transformata Fouriera stanowi podstawowe narzędzie analizy sygnałów, zarówno w teorii, jak i w praktycznych zastosowaniach w inżynierii, naukach przyrodniczych i ekonomii. DFT i FFT umożliwiają szybkie i wydajne przetwarzanie danych w systemach cyfrowych, od telefonii komórkowej po diagnostykę medyczną.


Time-Domain Signal
Amplitude
1.0
0.5
0.0
−0.5
−1.0
0.00
0.05
0.10
0.15
0.20
0.25
0.30
Time (s)


Frequency-Domain Representation (FFT)
Magnitude
1.0
0.8
0.6
0.4
0.2
0.0
0
100
200
300
400
500
Frequency (Hz)

Szereg Fouriera

- Szereg Fouriera to sposób przedstawiania funkcji okresowej jako sumy nieskończonej liczby funkcji sinusoidalnych (sinusów i kosinusów). Matematycznie, dla funkcji okresowej f(t) o okresie T, szereg Fouriera ma postać:

$$f(t) = a_0 + \sum_{n=1}^{\infty} (a_n \cos\left(\frac{2\pi nt}{T}\right) + b_n \sin\left(\frac{2\pi nt}{T}\right))$$

Gdzie:

a_0, a_n, b_n są współczynnikami Fouriera, obliczanymi jako całki funkcji pomnożonej przez odpowiednie funkcje trygonometryczne.

Transformata Fouriera

- Transformata Fouriera uogólnia ideę szeregu Fouriera dla funkcji nieokresowych, pozwalając przedstawić dowolną funkcję jako superpozycję fal sinusoidalnych o różnych częstotliwościach.

  - Transformata Fouriera funkcji f(t) definiowana jest wzorem:

$$F(\omega) = \int_{-\infty}^{+\infty} f(t) e^{-i\omega t}\, dt$$

Odwrotna transformata Fouriera umożliwia odzyskanie funkcji f(t) z F(ω)

$$f(t) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} F(\omega) e^{i\omega t}\, d\omega$$

DFT (Dyskretna Transformata Fouriera)

- DFT to wersja transformaty Fouriera dla danych dyskretnych (czyli dla skończonej liczby próbek sygnału).
- Wzór na DFT:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-\frac{i2\pi kn}{N}}, \ dla\ k = 0,1, \dots, N-1$$

  - x_n- wartość sygnału w chwili n
  - X_k- wartość odpowiadająca częstotliwości k,
  - N- liczba próbek

FFT (Szybka Transformata Fouriera)

FFT (Fast Fourier Transform) to szybki algorytm do obliczania DFT.

- Standardowe obliczenie DFT wymaga około N^2 operacji.
- FFT zmniejsza to do około $N \log N$ operacji.

Dlaczego to takie ważne?

- Jeśli mamy milon próbek sygnału:
  - DFT wymaga około 10^{12} operacji,
  - FFT wymaga tylko około 10^7 operacji — milion razy szybciej!

Najpopularniejszy algorytm FFT:

- Algorytm Cooleya-Tukeya (działa najlepiej, jeśli N jest potęgą 2: np. 256, 1024 itd.)

Gdzie używamy FFT?

- Kompresja muzyki i obrazów,
- Szybka analiza sygnałów w czasie rzeczywistym (np. podczas rozmów przez internet),
- W medycynie (np. MRI),
- W telekomunikacji (np. systemy 5G).

Przydatne linki

- Pochodne

1. https://pl.wikipedia.org/wiki/Pochodna_cz%C4%85stkowa
2. https://wyznacznik.pl/analiza/pochodne-funkcji-wielu-zmiennych
3. Etrapez.pl

- Inżynieria i testy

1. https://www.istqb.org/sdm_downloads/istqb-certified-tester-foundation-level-syllabus-v4-0/
2. https://www.youtube.com/watch?v=mqbi7V4Dd5M

- Transformata Fouriera

1. https://www.fourier-transform.com/
2. https://tutorial.math.lamar.edu/Classes/DE/FourierSeries.aspx

Dziękuje za uwagę

Mateusz Pękalak 2025r